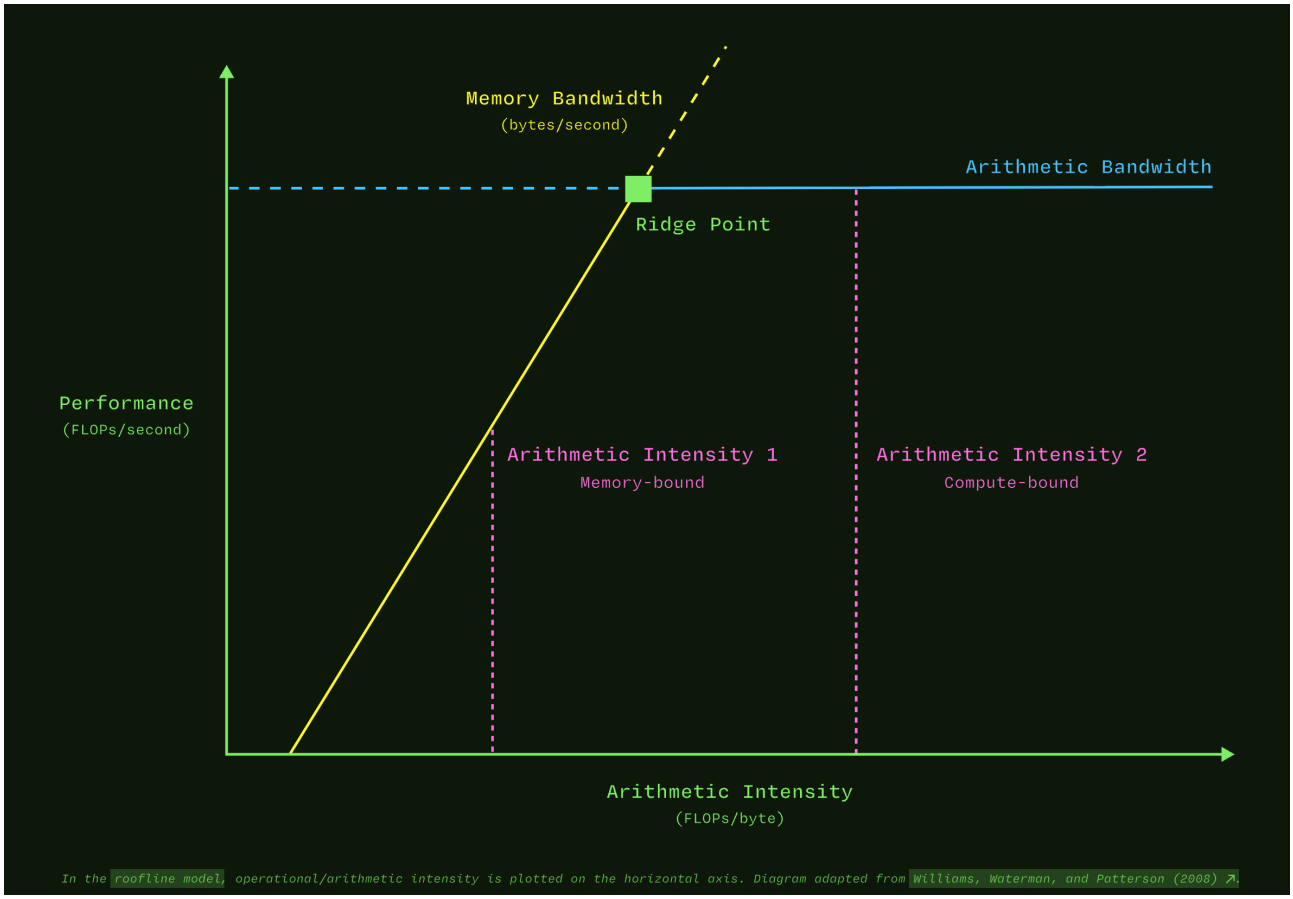
Roofline Analysis (루프라인 분석) 발표 자료

목차

1. 루프라인 분석이란?
2. GPU 성능 평가의 2가지 축
3. Arithmetic Intensity (산술 집약도)
4. 루프라인 모델 핵심 공식
5. 루프라인 그래프 읽는 법
6. 최신 GPU 아키텍처별 루프라인 비교
7. 세대별 루프라인 변화 트렌드
8. Memory-bound vs Compute-bound 최적화 전략
9. 핵심 정리

1. 루프라인 분석이란?



한 줄 요약

내 커널이 연산에 막혀있는지(Compute-bound), 메모리에 막혀있는지(Memory-bound) 판별하고, 올바른 최적화 방향을 선택하는 프레임워크

왜 필요한가?

GPU 최적화에서 가장 흔한 실수는 병목이 아닌 곳을 최적화하는 것입니다.

상황	잘못된 접근	올바른 접근
Memory-bound 커널	연산 로직 최적화	메모리 접근 패턴 개선
Compute-bound 커널	shared memory 추가	ILP 증가, fast math 활용

루프라인 분석은 이 판단을 정량적으로 해줍니다.

2. GPU 성능 평가의 2가지 축

GPU 성능을 이해하려면 두 가지 하드웨어 한계를 알아야 합니다.

축 1: Memory Bandwidth (메모리 대역폭)

- 정의: 초당 메모리에서 읽고/쓸 수 있는 데이터량 (TB/s)
- GPU가 아무리 빨라도, 데이터를 제때 공급하지 못하면 연산 유닛이 놀게 됩니다

축 2: Compute Throughput (연산 처리량)

- 정의: 초당 수행 가능한 부동소수점 연산 수 (TFLOPS)
- 데이터 타입(FP32/FP16/FP8)과 실행 유닛(CUDA Core vs Tensor Core)에 따라 크게 달라집니다

최신 GPU별 Peak 스펙 비교

GPU	세대	연도	메모리 BW (TB/s)	FP32 (TFLOPS)	FP16 TC (TFLOPS)	FP8 TC (TFLOPS)
A100 SXM	Ampere	2020	2.0	19.5	312	-
H100 SXM	Hopper	2022	3.35	67	1,979	3,958
H200 SXM	Hopper	2023	4.8	67	1,979	3,958
B200 SXM	Blackwell	2024	8.0	~85	2,250	4,500

핵심: 측정값이 "좋은지 나쁜지"는 반드시 peak 대비 비율로 판단해야 합니다.

예) 메모리 BW 1 TB/s 달성 → A100에서는 50%, B200에서는 12.5%

3. Arithmetic Intensity (산술 집약도)

정의

AI = 연산량(FLOPs) / 메모리 이동량(Bytes) 〔단위: FLOPs/Byte〕

"메모리에서 1바이트를 가져올 때, 평균적으로 몇 번의 연산을 수행하는가?"

직관적 이해

AI 값	의미	대표 연산
낮음	데이터를 많이 읽고, 연산은 적음	벡터 덧셈, RMS Norm, elementwise
중간	연산과 메모리 접근 균형	작은 GEMM, attention (small batch)
높음	데이터 재활용이 많고, 연산 집약적	대형 GEMM, convolution

4. 루프라인 모델 핵심 공식

달성 가능한 최대 성능 (Attainable Performance)

Performance = min(Peak FLOPs, AI × Peak Memory BW)

왜 min인가?

- AI × BW가 Peak FLOPs를 넘더라도, 실제로 하드웨어 연산 한계를 초과할 수 없기 때문

Transition Point (전환점)

전환점 AI = Peak FLOPs / Peak Memory BW

이 점이 Memory-bound와 Compute-bound의 경계입니다.

구간	조건	성능 결정 요인
전환점 왼쪽	AI 〈 전환점	메모리 대역폭이 성능을 결정 (Memory-bound)
전환점 오른쪽	AI 〉 전환점	연산 처리량이 성능을 결정 (Compute-bound)

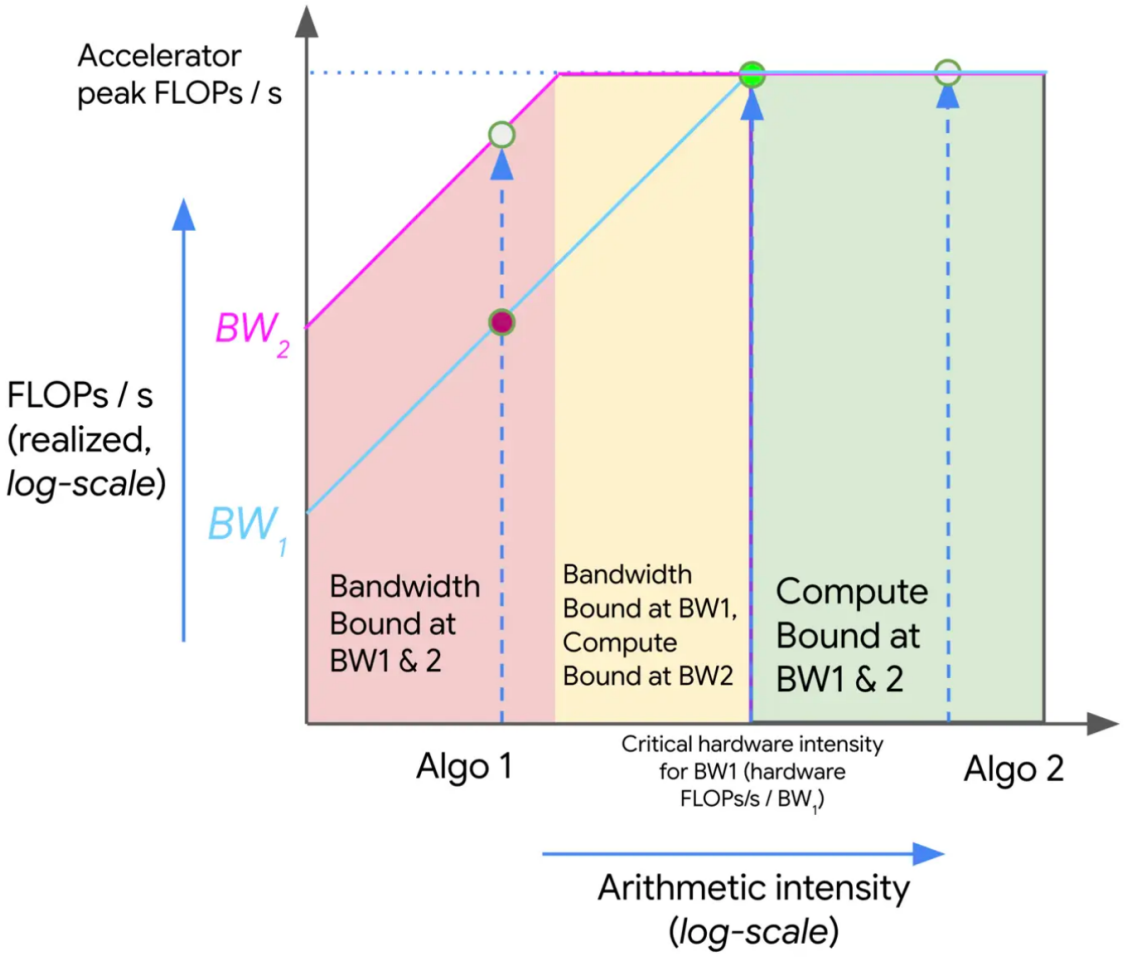
최신 GPU별 전환점 (Ridge Point)

GPU	FP32 전환점	FP16 TC 전환점	FP8 TC 전환점
A100	~10	~156	-
H100	~20	~591	~1,182
H200	~14	~412	~825
B200	~11	~281	~563

해석: H100의 FP16 TC 전환점은 591 FLOPs/Byte → AI가 591 이상이어야 Compute-bound
→ 대부분의 LLM 추론 연산은 Memory-bound

5. 루프라인 그래프 읽는 법

그래프 구조



그래프 읽기 포인트

1. 내 커널의 위치 확인: (AI, 실제 성능) 좌표를 찍습니다
2. 루프라인 경계와의 거리: 경계에 가까울수록 하드웨어를 잘 활용하는 것
3. 경계에서 멀다면: 최적화 여지가 남아있음

다중 루프라인 (Multi-Level Roofline)

Nsight Compute는 메모리 계층별로 여러 루프라인을 보여줍니다:

```
Performance (TFLOPS)
  ↑
  |                 ┌──────────── Peak FLOPs
  |          L1
  |
  |       L2
  |
  |  DRAM
  |
  |
  |──────────────────────────────→ AI (FLOPs/Byte)
```

레벨	대역폭 (H100 기준)	의미
DRAM	3.35 TB/s	HBM에서 데이터를 가져오는 속도
L2	~12 TB/s	L2 캐시에 히트할 때의 속도
L1	~33 TB/s	L1/Shared Memory에서의 속도

커널이 L2 루프라인에서는 경계 근처이지만 DRAM 루프라인에서는 멀리 떨어져 있다면,
L2 캐시 활용을 늘려서 DRAM 접근을 줄이는 것이 핵심 최적화 방향

6. 최신 GPU 아키텍처별 루프라인 비교

아키텍처별 상세 스펙

Ampere (A100, 2020)

항목	스펙
공정	TSMC 7nm
SM 수	108
L1/SM	192 KB (설정 가능)
L2 캐시	40 MB
메모리	80 GB HBM2e, 2.0 TB/s
특징	3세대 Tensor Core, 2:4 구조적 희소성, MIG

Hopper (H100/H200, 2022~2023)

항목	H100	H200
공정	TSMC 4N	TSMC 4N
SM 수	132	132 (동일 다이)
L1/SM	256 KB	256 KB
L2 캐시	50 MB	50 MB
메모리	80 GB HBM3, 3.35 TB/s	141 GB HBM3e, 4.8 TB/s
핵심 기능	TMA (Tensor Memory Accelerator)	HBM3e로 대역폭 43%↑

TMA (Tensor Memory Accelerator):

- SM 레지스터를 사용하지 않고 글로벌 → 공유 메모리로 비동기 5D 텐서 전송
- Warp Specialization 가능: 일부 warp는 데이터 이동, 일부는 연산 전담
- 메모리 roof와 연산 roof를 동시에 접근 가능

Blackwell (B200, 2024)

항목	스펙
공정	TSMC 4NP, 듀얼 다이 (2× GB100)

항목	스펙
트랜지스터	208B (10 TB/s NV-HBI로 연결)
SM 수	148
L1/SM	228 KB
TMEM/SM	256 KB (텐서 코어 전용 스크래치패드, 신규!)
L2 캐시	~96-126 MB (H100의 2.5배)
메모리	192 GB HBM3e, 8.0 TB/s
핵심 기능	5세대 TC, FP4, TMEM, NVLink 5.0 (1.8 TB/s)

TMEM (Tensor Memory):

- SM당 256 KB 전용 스크래치패드, 텐서 코어에 직결
- 유효 대역폭: SM당 ~16 TB/s read / ~8 TB/s write
- 148 SM 합계: ~100 TB/s 유효 TMEM 대역폭
- 새로운 메모리 계층 추가: HBM(8) → L2(20) → L1/SMEM(40) → TMEM(100) TB/s

세대별 핵심 비교표

지표	A100	H100	H200	B200
메모리 BW	2.0 TB/s	3.35 TB/s	4.8 TB/s	8.0 TB/s
FP16 TC	312 TFLOPS	1,979 TFLOPS	1,979 TFLOPS	2,250 TFLOPS
FP8 TC	-	3,958 TFLOPS	3,958 TFLOPS	4,500 TFLOPS
FP4 TC	-	-	-	9,000 TFLOPS
L2 캐시	40 MB	50 MB	50 MB	~96-126 MB
NVLink	600 GB/s	900 GB/s	900 GB/s	1,800 GB/s
TDP	400 W	700 W	700 W	1,000 W

7. 세대별 루프라인 변화 트렌드

대역폭 vs 연산 성장률

```
성장률 (A100 = 1.0x 기준)

        메모리 BW    FP16 TC 연산
A100 :  1.0x         1.0x
H100 :  1.68x        6.3x    ← 연산이 훨씬 빠르게 성장!
H200 :  2.4x         6.3x    ← 대역폭만 증가 (동일 다이)
B200 :  4.0x         7.2x    ← 대역폭이 따라잡는 중
```

전환점(Ridge Point) 변화와 그 의미

```
FP16 TC Ridge Point (FLOPs/Byte)

A100:  ■■■■■■■■■■■■■■■■ 156
H100:  ■■■■■■■■■■■■■■■■
```

```
B200: ■■■
```

목표: 연산 유닛 활용률을 peak에 가깝게

기법	설명	효과
ILP	독립 명령어를 늘려 파이프라인 활용	연산 stall 감소
Tensor Core 활용	TC 대응 데이터 타입 사용	연산 처리량 수배~수십배 증가
Fast Math	__fmaf_rn, intrinsic 함수	정확도 허용 시 연산 속도 향상
Occupancy 조정	레지스터/SMEM 사용량 최적화	SM 활용률 개선
Warp 스케줄링	의존성 최소화	연산 파이프라인 효율 증가

최적화 후 루프라인 위에서의 이동

```
Performance
  ↑
  |            ┌──────── Peak FLOPs
  |
  |          ●B (최적화 후)
  |         ↗
  |
  |       ●A (최적화 전)
  |
  |──────────────────────────→ AI
```

이동 방향	의미	사용 기법
↑ (수직 이동)	같은 AI에서 성능 향상	공유 메모리, bank conflict 제거, 더 나은 스케줄링
→ (수평 이동)	AI 증가 (연산 대비 메모리 접근 감소)	타일링, 커널 퓨전, 데이터 재활용
↗ (대각선 이동)	둘 다 개선	종합 최적화

9. 실전: NCU 프로파일링과 최적화 사례

Nsight Compute로 루프라인 분석하기

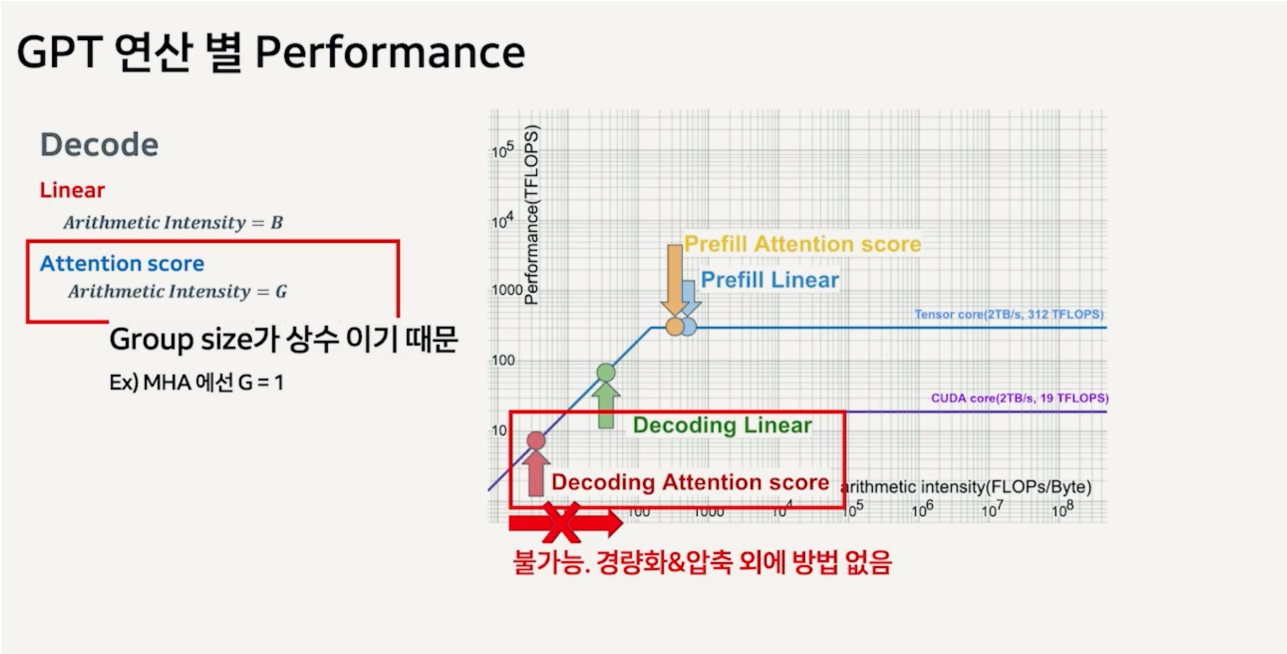
```
# 루프라인 메트릭 수집
ncu --set roofline -o profile_output ./my_kernel

# 특정 커널만 프로파일링 (vLLM 등 대형 앱에서 유용)
ncu --set roofline --kernel-name "attention" -o profile_output ./my_kernel
```

NCU는 SpeedOfLight 섹션에서 핵심 활용률을 보여줍니다:

메트릭	의미	병목 판단
SM [%]	연산 유닛 활용률	높으면 Compute-bound
Memory [%]	DRAM 대역폭 활용률	높으면 Memory-bound
L1 [%]	L1 캐시 처리량 활용률	높으면 L1에서 병목

사내 분석 영상



- https://tv.naver.com/v/67337608
- Encode, Decode 간의 산술집약도가 다르기 때문에 어떤 작업이냐에 따라 맞춤형 최적화가 필요하다.

9. 핵심 정리

루프라인 분석 3단계

```
Step 1: 내 커널의 AI 계산
  AI = FLOPs / Bytes
      ↓
Step 2: 전환점과 비교
  전환점 = Peak FLOPs / Peak BW
  AI &lt; 전환점 → Memory-bound
  AI &gt; 전환점 → Compute-bound
      ↓
Step 3: 올바른 방향으로 최적화
  Memory-bound → 메모리 접근 패턴 개선
  Compute-bound → 연산 효율 개선
```

최신 GPU에서의 핵심 트렌드

1. 연산이 대역폭보다 훨씬 빠르게 성장 → 대부분의 워크로드가 Memory-bound
2. 낮은 정밀도(FP8, FP4)가 필수 → Tensor Core 활용이 기본
3. 다중 메모리 계층(TMEM → L1 → L2 → HBM) → 각 계층에 맞는 최적화 필요
4. LLM 추론은 본질적으로 Memory-bound → 배치 크기, KV 캐시 관리가 핵심